



CS
2nd
material

Network

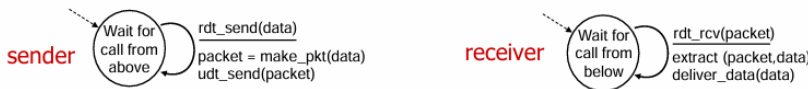


Summary on Lec 5

- استكمالا لآخر محاضره (٥٥)

rdt1.0: reliable transfer over a reliable channel

- underlying channel perfectly reliable
 - no bit errors
 - no loss of packets
- separate FSMs for sender, receiver:
 - sender sends data into underlying channel
 - receiver reads data from underlying channel



--> هنفترض الحاله المثاليه ان

التشائل بتاعتي Reliable

وبالتالي انا بفضل عند ال

sender مستني اول ما في

ابلكيشن بينده امر ال Send باخد

الباكيث واطلعهما زي ما هي

وابعت الباكيث دي علي التشائل

ف ال Receiver واقف مستني

امر من ال Layer ال تحته ف

يستقبل الباكيث ويعمل Extract للداثا من الباكيث ويبعت الداثا

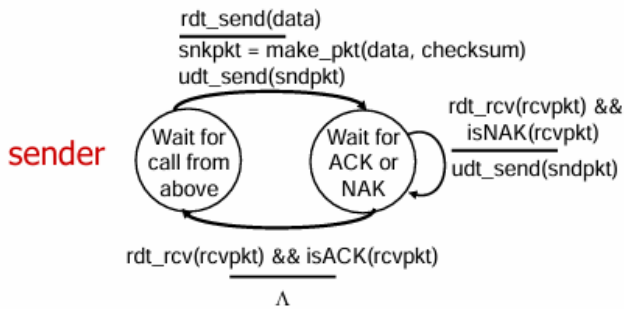
- هل بقي في حاجه اسمها reliable؟ هل في حاجه خاليه من الاخطاء؟ اكيد لا

طيب ازاي نتجنب الايروز دي؟ ب ان يتضاف CheckSum ف المسدج علشان المستلم

يقدر يعرف المسدج دي سليمه ولا مش سليمه ولازم يكون في فيدباك Positive

Acknowledgment يعني المستلم يقول للSender المسدج دي وصلت وتمام ويقدر بيعت ال

بعدها او Negative Acknowledgment يعني المسدج ال انت بعته دي فيها مشكله



- لما الباكيث تتبعت لازم بيقى

فيها الداثا وال CheckSum لان

دا ال هيقل للمستلم المسدج دي

سليمه ولا لا وبعدين لما الباكيث

تتبعت هيفضل مستني ال Ack

سواء كان بالايجاب او بالسلب

لو بالسلب ف هيبيعت الباكيث

تاني ويرجع يستني لو بالايجاب بيقى مش هيتاج يعمل حاجه تانيه

• طيب ايه الفرق بين rdt1.0 و rdt2.0 ان احنا في rdt2.0 ضيفنا ال CheckSum

وال Ack علشان نضمن وصول الداثا بشكل سليم

❖ بس طبعاً احنا من المحاضرات ال قبل كذا عارفين ان مفيش حاجه كامله 😊

- ايه بقي ال ممكن يحصل لو انا ك مستلم مستني الفيدباك وجالي بايظ او مش تمام؟

هضطر ابعث مرة ثانية بس المستلم هيعرف منين اذا كان دي الجديده ولا واحده اتبعنت ثاني؟ ف هنا ال Sender هيضيف مع كل مسدج حاجه اسمها ال sequence number (مش محتاج اكثر من 2-bit يبقوا ف الرقم دا دي 0 ي 1 يعني لو مرة اتبعنت 0 ال بعدها بيقى 1 وهكذا) ايه بقي دا؟ دا ال هيجدلي اذا كانت المسدج المبعوته دي جديدة ولا مبعوته لثاني مرة عن طريق اني اشوف ال sequence number دا مكرر وجالي قبل كدا ولا لاء دا بسميه

Rdt2.1

stop and wait

sender sends one packet, then waits for receiver response

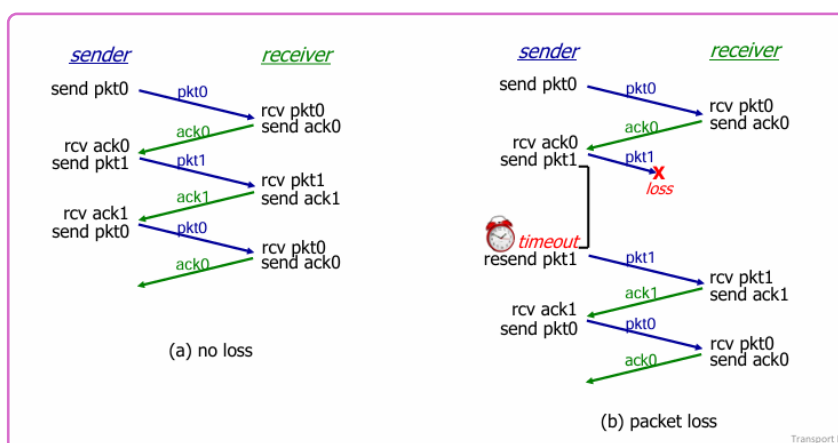
بمعني اني بيعت الباكيث واستنتي لحد ما يجيلي اجابه من المستلم لو كان بوزيتيف Ack هكمل واخذ ال بعده لو كان نيجاتيف او Corrupted هبعث ثاني

➤ rdt2.2: a NAK-free protocol :-

- هنا بقي لو جت باكيث سليمة هبعث ال Ack لو جت غلط بدل ما بيعتلي بوزيتيف ونيجاتيف هبعث ال Ack برودو بس ب Sequence Number غلط

➤ rdt3.0: channels with errors and loss:

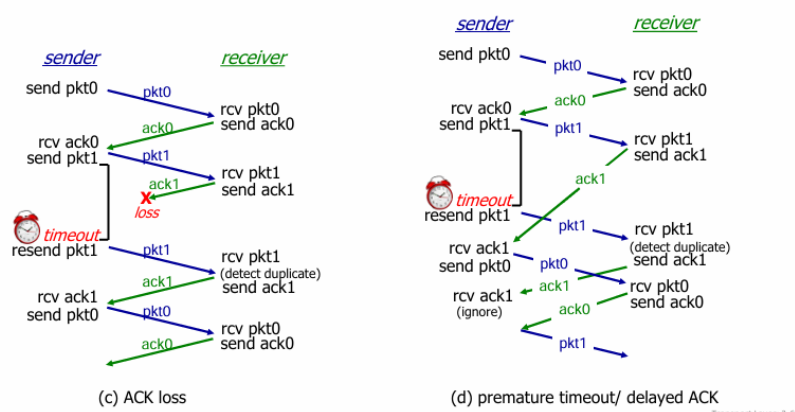
- الفكرة ببساطه ان ال Sender بيستني فترة معينه من الوقت علشان يستلم ال Ack من المستلم لو مجاش ال Ack دا ف الوقت دا هيعيد ارسال الرساله لو الرساله اتأخرت بس موصلتش في الوقت المحدد، الإعادة هتكون نسخة مكررة، لكن البروتوكول بيعرف يميزها عن الرساله الأصلية باستخدام ال Sequence Number وبيستخدم (timer) عشان يحدد الوقت "المعين" ده، ولو الوقت انتهى من غير ما يوصله ال Ack، بيعت الرساله ثاني.



- هنا الحاله (a):
ال Sender بيعت رساله (pkt0) للمستلم، والمستلم بيرد ب (ack0)، بعدها ال Sender بيعت الرساله اللي بعدها (pkt1) والمستلم يرد ب (ack1) العملية دي بتمشي من غير اي Loss
- الحاله (b): في الحاله

دي، ال Sender بيعت الرساله (pkt1)، لكن الرساله دي بتضيع (loss) ال Sender بيستني شوية، ولما مايجيش رد (Ack)، ال (timeout) بيشتغل ويعيد إرسال نفس الرساله (pkt1). بعدها المستلم يستقبلها ويرد ب (ack1)، والعملية تكمل عادي.

rdt3.0 in action



- هنا في الحالة (c):
ال Sender بيعت رسالة (pkt1) والمستلم بيستقبلها وبيبع (ack1)، لكن ال Ack ده بيحصله Loss في الطريق. ال Sender بيستنى شوية، ولما ال Ack ميوصلش، ال (timeout) بيشتغل ويعيد

إرسال الرسالة (pkt1) المستلم يستقبلها تاني ويعرف إنها مكررة، فبيرد بنفس ال (ack1)، والعملية تكمل بعدها عادي.

- ف الحالة (d): ال Sender بيعت رسالة (pkt1) والمستلم بيستقبلها وبيبع (ack1)، لكن ال Ack بيوصل متأخر، أو ال timeout في ال Sending بيتنتهي قبل ما ال Ack يوصل، فبيعيد إرسال الرسالة (pkt1) تاني. المستلم يستقبل الرسالة ويعرف إنها مكررة، فبيرد بنفس التأكيد (ack1) بس ال Sender بيتجاهله.

▪ U_{sender} : **utilization** – fraction of time sender busy sending

ال RTT هو ال
Propagation delay
رايح جاي يعني $2 * \text{propagation delay}$

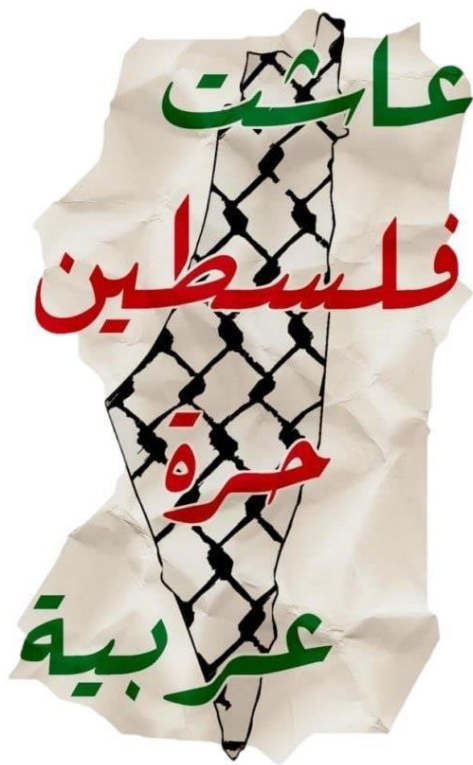
$$U_{sender} = \frac{L/R}{RTT + L/R}$$

- يبقى كذا ال rdt3.0 كويس وشغال وتوب التوب بس عيبه انه بياخد وقت طويل جدا جدا وهو بيستنى ال Ack علشان بيعت الباكيث ال وراها طيب نحل المشكلة دي ازاي؟
- مين قال اني لازم استنى؟ مانا ممكن بدل مابعت الباكيث الاولي واستني والتانيه واستني وهكذا ف انا كذا كذا عندي المسدج متقطعه علي هيئه باكيثس طب ما ابعتهم كلهم ورا بعض! ف علي ما الباكيث الاخيرة تتبعت مثلا تكون الاولي خلاص ال Ack بتاعها اتبعت ولا حاجه ف هنا انا مش بيعت One Packet at a Time انا بيعت N-Packet at a Time ف بنسميها Pipeline ولما بيجيلي ال Ack بستني لحد ما يجيلي ال n-Acks لو جم هبعت ال N ال بعدهم لو مجوش هشوف الباكيث ال وقعت فين وابتدي ابعت من عندها تاني للآخر ودا بنسميه Go-Back-N بالتالي ال Utilization هيبقي

3 دي ال هي N-
Packets يعني متغيرة
علي حسب انا بيعت قد
ايه

$$U_{sender} = \frac{3L/R}{RTT + L/R}$$

- طيب نوضح ال Go-Back-N وهو اني بعت N-Packets ال N دول في شوية منهم جالهم Ack وشوية لسه ف اول ما بيحصل timeout بشوف اخر باكيث جالها Ack بالترتيب ايه وابتدي من ال بعدها وابتعت تاني.



لا تنسو الدعاء لآخواننا في
فلسطين و غزة و العراق و
لبنان و سوريا و السودان.
و تذكر دائما المقاطعه ولا
تتخلي عن مبادئك و انت بتجاهد
معاهم بمقاطعتك.